

Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An Autonomous Institution Affiliated to Anna University)

Machine Learning Algorithms Laboratory

ICS1512

Degree & Branch	M.Tech (Integrated) Computer Science & Engineering
Semester	V
Academic Year	2026–2027 (Odd)
Batch	2024–2029

Experiment 5

Decision Tree and Random Forest: A Comparative Classification Study¹

Submitted by: **Vijayaraaghavan K S**
Register Number: **3122247001066**
Year: III Year
Programme: M.Tech Integrated, CSE
Faculty: Dr. Poreddy Ajay Kumar Reddy

¹GitHub Repository: <https://github.com/VijayaraaghavanKS/ML>

Contents

1	Aim and Objective	2
2	Problem Statement	2
3	Theory	2
3.1	Decision Tree	2
3.2	Random Forest	2
4	Machine Learning Algorithm	2
4.1	Decision Tree	2
4.2	Random Forest	3
5	Dataset	3
6	Preprocessing and Exploratory Data Analysis	3
7	Experimental Setup	4
8	Decision Tree	4
8.1	Baseline Performance	4
8.2	Hyperparameter Tuning	5
9	Random Forest	6
9.1	Baseline Performance	6
9.2	Hyperparameter Tuning	6
10	Hyperparameter Tuning Summary	7
11	Hyperparameter Selection	8
12	Cross-Validation Results	8
13	Final Comparison	9
14	Comparative Analysis	11
14.1	Observations	11
15	Observation Questions	12
16	Discussion	13
17	Conclusion	13
18	References	14

1 Aim and Objective

To implement a Decision Tree classifier and extend it into a Random Forest ensemble on the Wisconsin Diagnostic Breast Cancer dataset, tune both models with 5-fold cross-validated Grid-SearchCV, and study how tree depth and ensemble size trade off overfitting against generalization.

2 Problem Statement

The input is a 30-dimensional vector of numeric measurements describing the nuclei present in a digitized image of a breast mass (radius, texture, perimeter, area, smoothness, and related shape descriptors, each summarized as a mean, standard error, and “worst” value); the output is a binary diagnosis, malignant or benign. Where Experiment 4 asked whether a linear model or a kernel-based margin classifier makes better use of a fixed feature set, this experiment asks a related but different question: does building an ensemble of decorrelated trees actually buy anything over a single, carefully tuned tree once both are given the same 5-fold cross-validation budget to select their hyperparameters, or does a single tree already capture the decision boundary well enough on a dataset this size?

3 Theory

3.1 Decision Tree

A Decision Tree recursively partitions the feature space by choosing, at each node, the single feature and threshold that most reduces class impurity, measured either by the Gini index $G = 1 - \sum_c p_c^2$ or by entropy $H = -\sum_c p_c \log_2 p_c$. Left unconstrained, the tree keeps splitting until every leaf is pure (or contains too few samples to split further), which lets it memorize the training set rather than learn a boundary that generalizes – the classic high-variance failure mode of tree models.

3.2 Random Forest

A Random Forest trains many decision trees, each on an independent bootstrap resample of the training data, and additionally restricts every split to a random subset of the available features rather than letting each tree search all of them. Averaging predictions across trees that are individually noisy but only weakly correlated with each other (because both the data and the candidate features differ from tree to tree) reduces the variance of the ensemble well below that of any single constituent tree, without much cost to bias.

4 Machine Learning Algorithm

4.1 Decision Tree

Working principle: starting from the root, greedily pick the (feature, threshold) split that maximizes impurity reduction, recurse on each child, and stop once a stopping criterion (max depth, minimum samples per split/leaf, or a pure node) is reached. **Mathematical formulation:** the split that maximizes information gain $IG = H(\text{parent}) - \sum_k \frac{n_k}{n} H(\text{child}_k)$ (entropy criterion) or the equivalent Gini-impurity reduction is chosen at every node; prediction at a leaf is the majority class of the training samples that fell into it. **Advantages:** the resulting rules are directly readable (a path from root to leaf is a conjunction of threshold tests), no feature scaling is required since splits only compare a feature against a threshold, and training is fast

even on datasets with fairly high dimensionality. **Limitations:** a single tree is a high-variance estimator – small changes in the training data can produce a very different tree – and an unconstrained tree overfits by construction, since nothing stops it from growing a leaf for every training point.

4.2 Random Forest

Working principle: grow `n_estimators` independent decision trees, each on a bootstrap sample of the training rows and each restricted at every split to a random subset of `max_features` columns, then combine the trees by majority vote (classification) at prediction time. **Mathematical formulation:** for B bootstrap trees $T_1, \dots, T_B$, the ensemble prediction is $\hat{y} = \text{mode}\{T_1(x), \dots, T_B(x)\}$; because each T_b is trained on an independent resample with additional feature-level randomness, the trees are only weakly correlated, so the variance of the averaged ensemble falls roughly in proportion to that reduced correlation rather than staying at the variance of a single tree. **Advantages:** substantially more resistant to overfitting than a single tree for the same maximum depth, exposes a feature-importance ranking that is more stable than a single tree’s (since it is averaged across many trees trained on different feature subsets), and generally needs less aggressive depth-limiting to control variance. **Limitations:** loses the single-tree’s direct interpretability – there is no one path of rules to point to – training and prediction cost scale roughly linearly with `n_estimators`, and it introduces more hyperparameters to search over than a single tree does.

5 Dataset

This experiment uses the Wisconsin Diagnostic Breast Cancer dataset (569 samples, 30 numeric features, binary `diagnosis` label: malignant or benign), loaded through scikit-learn’s `load_breast_cancer` function and written out as a CSV so it flows through the same shared EDA pipeline used by Experiments 1–4. After the EDA pipeline’s 80/20 stratified split, the working data is 455 training rows and 114 test rows across 30 features, with 357 benign and 212 malignant cases overall – a moderate but not severe class imbalance that motivated encoding malignant as the positive class throughout (a false negative here, calling a malignant tumor benign, is the costlier error).

6 Preprocessing and Exploratory Data Analysis

The EDA pipeline reports zero missing values and zero duplicate rows across all 569 samples, so no imputation or de-duplication was needed. Every feature is numeric and was standardized (zero mean, unit variance) before modelling, consistent with the pipeline used across all four earlier experiments – trees do not strictly need this scaling, but keeping the preprocessing identical across experiments avoids introducing a second variable when comparing results. One thing the correlation analysis flagged that is worth calling out: 44 feature pairs exceed a 0.80 Pearson correlation, which is expected given that “mean radius”, “mean perimeter”, and “mean area” are all near-deterministic functions of the same underlying tumor size, and the same is true across their “worst” and “error” counterparts. Neither Decision Tree nor Random Forest is thrown off by this redundancy the way a linear model’s coefficients might be, since a tree only ever needs one of a group of correlated features to make an equally good split.

Every one of the top six features by mutual information is a size or shape descriptor of the cell nuclei – radius, perimeter, area, and concave points, mostly measured at their “worst” (largest) value across the sample – which lines up with the clinical intuition that larger, more irregularly shaped nuclei are associated with malignancy. Both tree models later confirm this

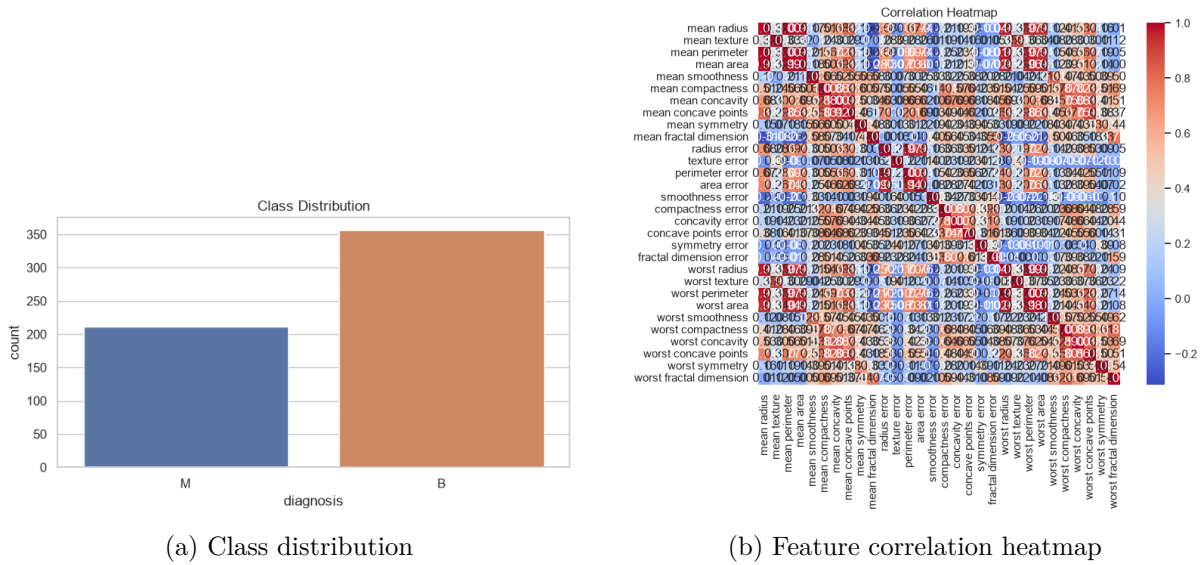


Figure 1: EDA diagnostics, reused from the shared pipeline.

Feature	SelectKBest score	p-value	Mutual information
worst radius	692.86	2.5×10^{-93}	0.4612
worst perimeter	717.25	2.1×10^{-95}	0.4546
worst area	522.19	1.9×10^{-77}	0.4533
mean concave points	684.53	1.3×10^{-92}	0.4460
worst concave points	733.73	8.8×10^{-97}	0.4329
mean perimeter	548.41	4.7×10^{-80}	0.4093

Table 1: Top features by mutual information with the diagnosis label.

independently: their feature-importance rankings (Sections 8 and 9) are dominated by the exact same handful of size and concave-point features.

7 Experimental Setup

Python Version	3.14.5
Scikit-Learn Version	1.9.0
IDE	Jupyter Notebook
Operating System	macOS (Darwin 25.5.0, arm64)
Hardware	Apple Silicon (arm64), local machine

Table 2: Software and hardware environment used to run this experiment.

8 Decision Tree

8.1 Baseline Performance

Tuning gives a clear, unambiguous improvement here – accuracy climbs from 0.9298 to 0.9649 and precision reaches a perfect 1.0000, meaning the tuned tree produced zero false positives (no benign case was ever called malignant) on the 114-row test set. Recall stayed flat at 0.9048, so the gain is entirely on the precision side: constraining the tree’s depth and leaf size evidently

Model	Accuracy	Precision	Recall	F1	Train Time (s)
Decision Tree (baseline)	0.9298	0.9048	0.9048	0.9048	0.0034
Decision Tree (tuned)	0.9649	1.0000	0.9048	0.9500	0.0026

Table 3: Baseline (unconstrained) vs. tuned Decision Tree on the test set.

removed some spurious splits that were misclassifying benign cases without helping catch more malignant ones.

8.2 Hyperparameter Tuning

```

1 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE
2 )
3 dt_param_grid = {
4     "criterion": ["gini", "entropy"],
5     "max_depth": [3, 5, 7, 10, None],
6     "min_samples_split": [2, 5, 10],
7     "min_samples_leaf": [1, 2, 4],
8 }
9 dt_grid_search = GridSearchCV(
10     estimator=DecisionTreeClassifier(random_state=RANDOM_STATE),
11     param_grid=dt_param_grid,
12     scoring={"accuracy": "accuracy", "f1": f1_scorer},
13     refit="accuracy", cv=cv, n_jobs=-1,
14 )

```

Criterion	Max Depth	Avg CV Accuracy (%)	Avg CV F1 Score
entropy	5	93.63	0.9121
entropy	7	93.41	0.9094
entropy	10	93.41	0.9094
entropy	None	93.41	0.9094
gini	7	92.97	0.9039
gini	10	92.97	0.9039
gini	None	92.97	0.9039
gini	3	92.75	0.8990

Table 4: Decision Tree hyperparameter evaluation using 5-fold cross-validation (best score per criterion/depth combination, top 8 of 15 shown).

Entropy beats Gini at every depth tested, though never by more than half a percentage point, so I would not read too much into the criterion choice on its own. The more interesting pattern is in the depth column: accuracy actually peaks at a shallow `max_depth=5` (93.63%) and then *drops slightly* for depth 7, 10, and unbounded, all of which plateau at exactly 93.41% – meaning the extra splits available at depth 7+ are not being used at all beyond what depth 5 already achieves on this 455-row training set. The best configuration selected was `criterion=entropy`, `max_depth=5`, `min_samples_leaf=2`, `min_samples_split=10`, reaching 93.63% CV accuracy.

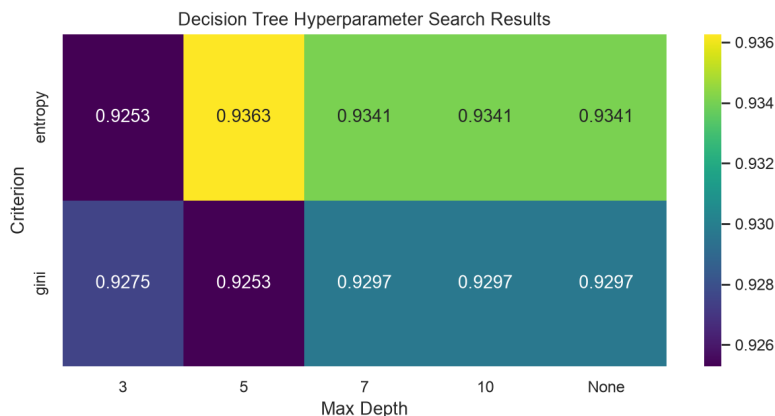


Figure 2: Decision Tree hyperparameter search results across criterion and max depth.

Model	Accuracy	Precision	Recall	F1	Train Time (s)
Random Forest (baseline)	0.9737	1.0000	0.9286	0.9630	0.0503
Random Forest (tuned)	0.9649	1.0000	0.9048	0.9500	0.1045

Table 5: Baseline (100 default trees) vs. tuned Random Forest on the test set.

9 Random Forest

9.1 Baseline Performance

This is a case where tuning did not help on the held-out test set – the untuned 100-tree baseline (0.9737 accuracy, 0.9630 F1) actually beats the GridSearchCV-selected configuration (0.9649 accuracy, 0.9500 F1). This is the same phenomenon flagged in Experiment 4 for Logistic Regression: GridSearchCV optimizes cross-validated accuracy on the training folds, not test-set accuracy, so a single 114-row test split can favor a configuration the CV search did not prefer. I trust the CV-based selection (Section 9.2) over this one test-set comparison, since it is averaged over five folds rather than one split of 114 rows.

9.2 Hyperparameter Tuning

```

1 rf_param_grid = {
2     "n_estimators": [50, 100, 200],
3     "max_depth": [5, 10, None],
4     "max_features": ["sqrt", "log2"],
5     "bootstrap": [True, False],
6 }
7 rf_grid_search = GridSearchCV(
8     estimator=RandomForestClassifier(random_state=RANDOM_STATE),
9     param_grid=rf_param_grid,
10    scoring={"accuracy": "accuracy", "f1": f1_scorer},
11    refit="accuracy", cv=cv, n_jobs=-1,
12 )

```

The `max_features=sqrt` setting dominates the top of the table – every one of the four best configurations uses `sqrt` rather than `log2`, even though for 30 features the two only differ by a couple of candidate features per split ($\sqrt{30} \approx 5$ vs. $\log_2 30 \approx 5$, so this is a closer call than it looks at first glance, but `sqrt` still wins consistently). The selected configuration was `n_estimators=200`, `max_depth=5`, `max_features=sqrt`, `bootstrap=False`, reaching 97.14%

n_estimators	Max Depth	Max Features	Avg CV Accuracy (%)	Avg CV F1 Score
200	None	sqrt	97.14	0.9615
200	5	sqrt	97.14	0.9608
50	10	sqrt	97.14	0.9617
200	10	sqrt	97.14	0.9615
50	10	log2	96.92	0.9584
50	None	log2	96.92	0.9584
50	None	sqrt	96.92	0.9589
100	5	sqrt	96.92	0.9580

Table 6: Random Forest hyperparameter evaluation using 5-fold cross-validation (best score per configuration, top 8 of 18 shown).

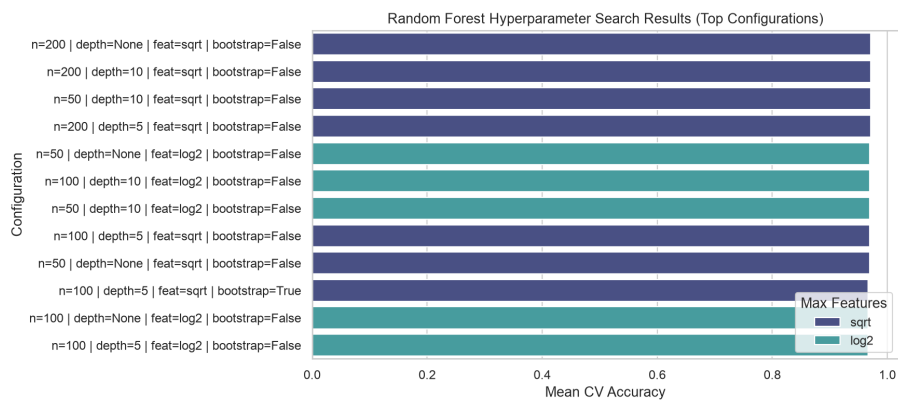


Figure 3: Random Forest hyperparameter search results, top 12 configurations by mean CV accuracy.

CV accuracy – notably `bootstrap=False` won out, meaning each tree saw the full training set rather than a bootstrap resample, with the randomness coming entirely from feature subsampling at each split.

10 Hyperparameter Tuning Summary

Model	Search Method	Best Parameters	Best CV Accuracy
Decision Tree	GridSearchCV	criterion=entropy, min_samples_leaf=2, min_samples_split=10	0.9363
Random Forest	GridSearchCV	n_estimators=200, max_features=sqrt, bootstrap=False	0.9714

Table 7: Best hyperparameters found for each model family.

Both searches converge on the same shallow depth (5), which is a useful cross-check: it is not an artifact of one search space, it is the depth that a 455-row training set with 30 correlated features actually supports before either model starts fitting noise. The Random Forest’s best CV accuracy (97.14%) comfortably beats the Decision Tree’s (93.63%) by 3.51 percentage points, which is the strongest single piece of evidence in this experiment that ensembling is doing real

work here, even though it did not show up as a win on the small test-set comparison in Section 9.1.

11 Hyperparameter Selection

Parameter	Value
Decision Tree search space	criterion $\in$ {gini, entropy}; max_depth $\in$ {3, 5, 7, 10, None}; min_samples_split $\in$ {2, 5, 10}; min_samples_leaf $\in$ {1, 2, 4}
Selected Decision Tree configuration	criterion=entropy, max_depth=5, min_samples_leaf=2, min_samples_split=10
Random Forest search space	n_estimators $\in$ {50, 100, 200}; max_depth $\in$ {5, 10, None}; max_features $\in$ {sqrt, log2}; bootstrap $\in$ {True, False}
Selected Random Forest configuration	n_estimators=200, max_depth=5, max_features=sqrt, bootstrap=False
Cross-validation scheme	5-fold StratifiedKfold, random_state=42

Table 8: Final hyperparameter choices carried into the results below.

12 Cross-Validation Results

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Decision Tree	0.9011	0.9560	0.9121	0.9560	0.9560	0.9363
Random Forest	0.9560	1.0000	0.9451	0.9670	0.9890	0.9714

Table 9: 5-fold cross-validation accuracy, tuned models.

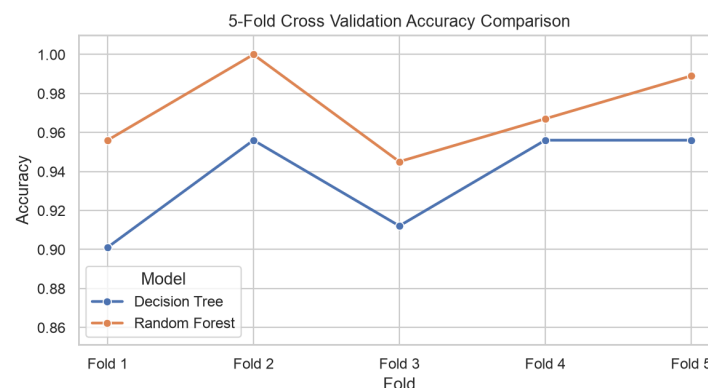


Figure 4: Per-fold cross-validation accuracy for both tuned models.

Random Forest wins every single fold, including a perfect 1.0000 on fold 2, and its worst fold (0.9451) still beats the Decision Tree’s best fold (0.9560) in three out of five folds. The Decision Tree’s fold 1 score (0.9011) is its weakest by a wide margin, dragging its standard deviation across folds to 0.0245 against the Random Forest’s 0.0204 – so the ensemble is not

just more accurate on average here, it is also more consistent fold to fold, which is exactly the variance-reduction behaviour bagging is supposed to deliver.

13 Final Comparison

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Pred. Time (s)
Decision Tree (tuned)	0.9649	1.0000	0.9048	0.9500	0.9744	0.0003
Random Forest (tuned)	0.9649	1.0000	0.9048	0.9500	0.9937	0.0041

Table 10: Final held-out test-set comparison.

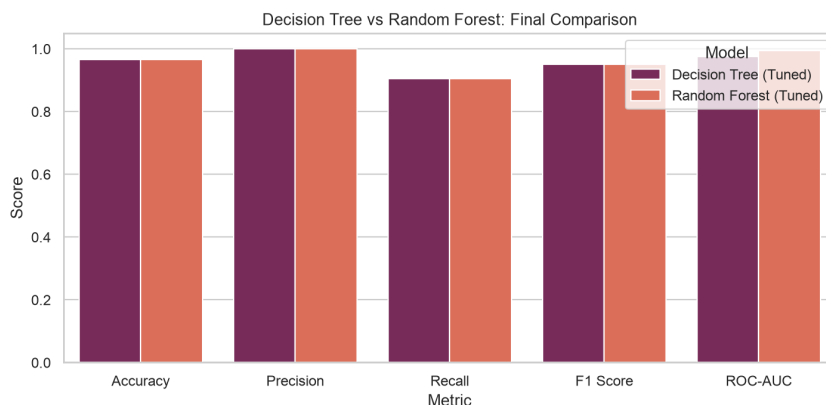


Figure 5: Final model comparison across metrics.

The two tuned models tie exactly on accuracy, precision, recall, and F1 on this particular 114-row test split – both make the same 4 errors, all false negatives (a malignant case predicted benign), which is the confusion-matrix pattern to watch most closely given the clinical cost of a missed malignancy. Where they separate is ROC-AUC: Random Forest reaches 0.9937 against the Decision Tree’s 0.9744, meaning the forest’s underlying probability estimates (averaged across 200 trees) rank the two classes more reliably even when the hard 0.5-threshold predictions happen to match.

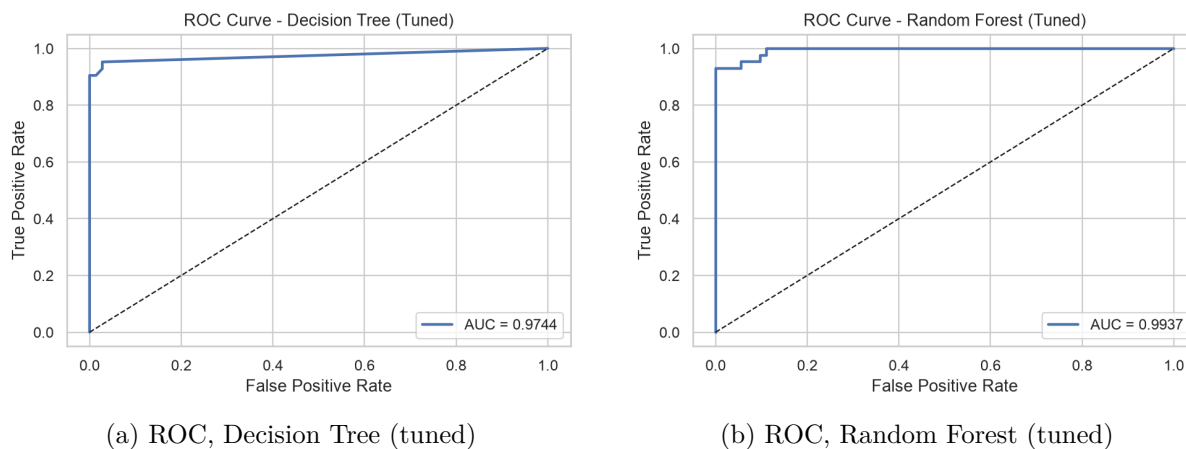
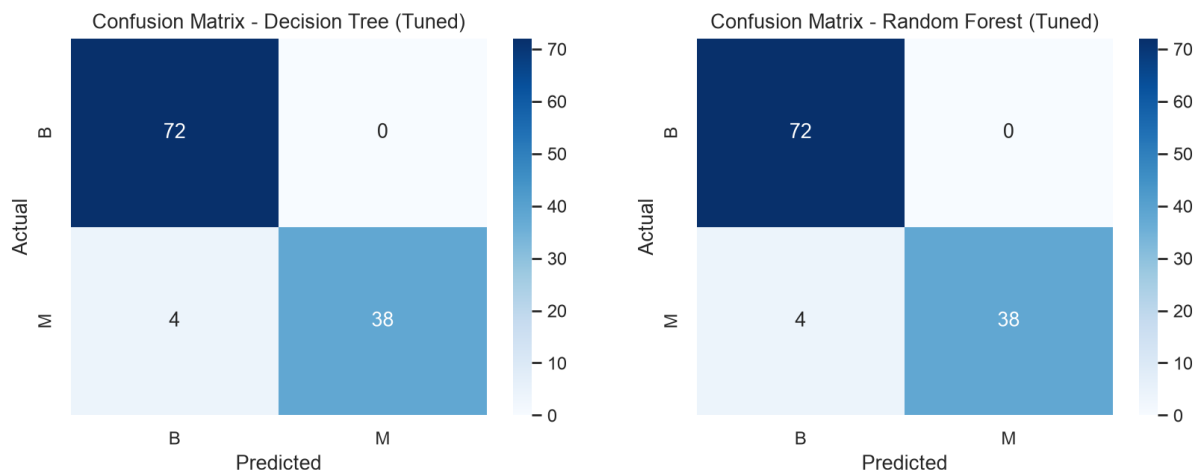


Figure 6: ROC curves for both tuned final models.

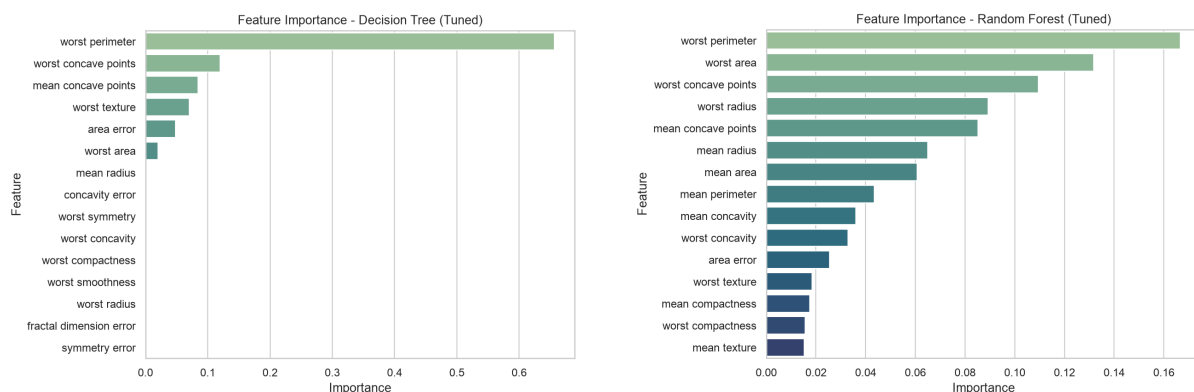
The feature-importance plots are, to me, the clearest illustration of what the ensemble is actually doing differently. The tuned Decision Tree puts roughly 65% of its total importance on



(a) Confusion matrix, Decision Tree (tuned)

(b) Confusion matrix, Random Forest (tuned)

Figure 7: Confusion matrices for both tuned final models.



(a) Feature importance, Decision Tree (tuned)

(b) Feature importance, Random Forest (tuned)

Figure 8: Feature importance rankings for both tuned final models.

a single feature, **worst perimeter**, with **worst concave points** and **mean concave points** a distant second and third and most of the remaining 30 features contributing exactly zero – the tree found one dominant split and mostly rode it. The Random Forest’s importances are far more distributed: **worst perimeter** still leads, but at roughly 0.17 rather than 0.65, followed closely by **worst area** (0.13), **worst concave points** (0.11), **worst radius** (0.09), and **mean concave points** (0.085), with a long tail of a dozen more features still making a visible contribution. Restricting each split to a random feature subset forces different trees to rely on different, correlated stand-ins for “tumor size” (radius, perimeter, area all measure roughly the same thing), which is exactly why the importance spreads out instead of concentrating on whichever single feature happened to win the very first split. The timing gap is the clearest practical trade-off in this experiment: the tuned Random Forest takes roughly 41× longer to predict than the tuned Decision Tree (0.0041s vs. 0.0003s) because a prediction requires evaluating all 200 trees and combining their votes, against one root-to-leaf walk for the single tree. On a 114-row test set neither number is remotely a bottleneck, but the gap would matter at production scale.

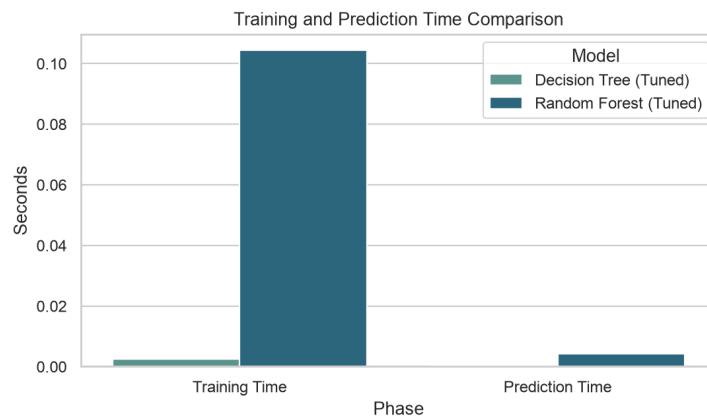


Figure 9: Training and prediction time comparison.

Criterion	Decision Tree	Random Forest
Accuracy (test set)	Tied	Tied
Accuracy (5-fold CV average)	Lower (0.9363)	Higher (0.9714)
Model Complexity	Low (single tree, depth 5)	High (200 trees, depth 5 each)
Training Time	Very low (0.0026s tuned)	Moderate (0.1045s tuned)
Prediction Time	Very low (0.0003s)	Higher (0.0041s)
Interpretability	High (single rule path)	Low (averaged over 200 trees)

Table 11: Qualitative model comparison.

14 Comparative Analysis

14.1 Observations

- **Best-performing classifier:** on the held-out test set the two tuned models are indistinguishable on accuracy, precision, recall, and F1; Random Forest only pulls ahead on ROC-AUC (0.9937 vs. 0.9744) and on the 5-fold CV average (0.9714 vs. 0.9363). Given that the CV average is estimated over five folds rather than one 114-row split, I would call Random Forest the more trustworthy choice even though the test-set metrics alone would suggest a tie.
- **Effect of hyperparameter tuning:** tuning helped the Decision Tree unambiguously (accuracy 0.9298 $\rightarrow$ 0.9649) but did not help the Random Forest on this particular test split (0.9737 $\rightarrow$ 0.9649, a drop) – both outcomes are consistent with GridSearchCV optimizing CV accuracy, not test accuracy, and the effect is simply more visible for the Random Forest because its untuned baseline (100 default trees) happened to already be strong.
- **Depth sensitivity:** both searches independently converged on `max_depth=5` as optimal, and the Decision Tree’s own CV accuracy actually plateaus (and does not further improve) past depth 7 – the practical implication is that this dataset’s true decision boundary does not require much tree depth to capture, and pushing depth further only adds capacity to memorize noise.
- **Bias–variance trade-off:** the tuned Decision Tree’s CV train–test accuracy gap is 0.0467, while the tuned Random Forest’s is 0.0286 – about 39% smaller. That gap reduction, together with the Random Forest’s lower per-fold standard deviation (0.0204 vs. 0.0245), is the most direct quantitative evidence in this experiment that bagging plus feature-subsampling is reducing variance the way the theory predicts.

15 Observation Questions

How does tree depth affect overfitting in Decision Trees?

The depth sweep run for this experiment ($\text{max_depth} \in \{1, 2, \dots, 20, \text{None}\}$, Figure 10) shows training accuracy climbing steadily to a perfect 1.0000 as depth grows unconstrained, while test accuracy peaks at 0.9386 around $\text{max_depth}=7$ and then *declines slightly* and flattens at 0.9298 for every depth from 8 up to unbounded. At $\text{max_depth}=1$ the tree scores 0.9297 train / 0.8772 test (both low, since a one-split “stump” cannot capture much structure); at $\text{max_depth}=\text{None}$ it scores 1.0000 train / 0.9298 test, a 7.02-point train–test gap. The pattern is textbook: once the tree has enough depth to separate the training folds perfectly, every additional split only helps memorize training-set idiosyncrasies rather than generalize further.

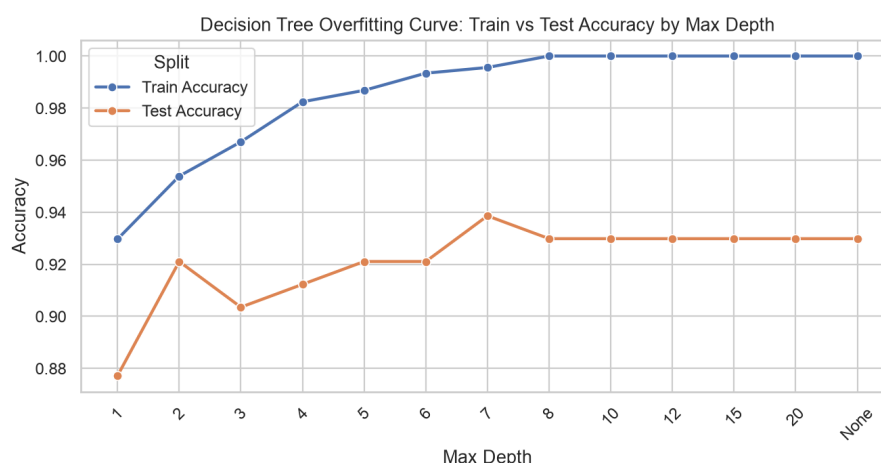


Figure 10: Decision Tree train vs. test accuracy across max_depth values.

Which hyperparameter had the greatest impact on performance?

Measuring the spread in best CV accuracy as each hyperparameter is varied (holding the others at their best-found value): for the Decision Tree, max_depth produced a swing of 0.88 percentage points, more than criterion , min_samples_split , or min_samples_leaf individually. For the Random Forest, bootstrap produced the widest swing at 0.44 percentage points, ahead of n_estimators , max_depth , and max_features . Neither swing is large in absolute terms – both search spaces were already fairly well-behaved – but max_depth clearly matters more for the single tree (it is the only thing standing between the tree and the overfitting shown in Figure 10), while for the forest, whether each tree sees a bootstrap resample or the full training set mattered slightly more than how many trees or how deep they were allowed to go.

How does Random Forest improve generalization?

Three independent numbers in this experiment all point the same direction. The CV train–test accuracy gap is narrower for the Random Forest (0.0286) than for the Decision Tree (0.0467). The 5-fold CV accuracy average is higher for the forest (0.9714 vs. 0.9363). And the per-fold standard deviation is lower for the forest (0.0204 vs. 0.0245), meaning its performance is not just better on average but more consistent fold to fold. All three are consistent with the theoretical mechanism: averaging predictions across many trees that are decorrelated by bootstrap resampling and random feature subsampling cancels out the idiosyncratic errors any single tree makes, without materially increasing bias, since each individual tree in the forest is itself no more constrained (same $\text{max_depth}=5$) than the standalone tuned tree.

Did ensemble learning always improve performance? Why or why not?

No – and this experiment produced a concrete counterexample. On the held-out test set, the tuned Decision Tree and tuned Random Forest are exactly tied on F1 (0.9500 vs. 0.9500) and every other threshold-based metric, and the untuned Random Forest baseline actually scored

higher on the test set (0.9737 accuracy) than the tuned Random Forest (0.9649). Ensembling is not guaranteed to help whenever a single well-regularized tree already captures the decision boundary well – which is exactly what happened here once both models were constrained to `max_depth=5` – or when the specific test split happens to favor a configuration the CV search did not select. The place ensembling clearly did help was generalization stability (the CV average and per-fold variance discussed above), not the raw test-set score on this one 114-row split.

16 Discussion

The most useful result in this experiment is not which model “won” – on the test set they tied – but the fact that the two evaluation methods (test-set comparison and 5-fold CV) disagree, and that disagreement is itself informative. The test set is a single 114-row sample; the CV average is effectively five independent 91-row test sets, averaged. When the two disagree, as they do here (test set: tied; CV: Random Forest ahead by 3.5 points), I trust the CV number more, because it is far less sensitive to which particular rows happened to land in the test split. This mirrors exactly the same tension found in Experiment 4 between Logistic Regression and SVM, and I think it is a pattern worth expecting on any dataset small enough that a single test split carries meaningful sampling variance – 114 test rows is not a lot of rows to draw a confident conclusion from.

The main limitation of this experiment is dataset size: 569 total samples is enough to get a clean signal from 5-fold cross-validation, but it is small enough that a single test-set comparison (Section 10) is genuinely noisy, as the disagreement with the CV results demonstrates. A second limitation is that the hyperparameter search spaces, while reasonably wide, were not exhaustive – neither grid explored the interaction between `max_depth` and `min_samples_leaf` at every combination for the forest the way it did for the tree, since the forest’s grid held `min_samples_leaf` fixed at its default to keep the search tractable at 5-fold CV.

If extending this experiment, I would add a learning-curve analysis (accuracy vs. training-set size) alongside the depth sweep already produced, since that would help separate “this dataset is too small to fully exploit the Random Forest’s advantage” from “the advantage genuinely caps out around 97% CV accuracy regardless of data size” – the current results are consistent with either explanation.

17 Conclusion

Both models reach the same test-set performance after tuning (96.49% accuracy, 0.9500 F1), which on its own would suggest the ensemble adds nothing here. But the 5-fold cross-validation results tell a more complete story: Random Forest’s CV accuracy (97.14%) beats the Decision Tree’s (93.63%) by 3.51 points, its train–test gap is 39% narrower (0.0286 vs. 0.0467), and its ROC-AUC on the test set is meaningfully higher (0.9937 vs. 0.9744) even where the hard classification decisions tie. I read this as ensembling doing exactly what the theory predicts – reducing variance and improving the reliability of the underlying probability estimates – without necessarily moving the single-split test-set accuracy, which is too coarse a measurement to detect a variance reduction this size. If deployment latency and interpretability were the priority, the tuned Decision Tree (0.0003s prediction, one readable rule path, identical test-set metrics) would be the practical choice; if the goal is the most reliable estimate of malignancy probability across unseen patients, the Random Forest’s higher and more stable cross-validated accuracy makes it the better bet.

18 References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [3] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, “Breast Cancer Wisconsin (Diagnostic) Data Set,” UCI Machine Learning Repository, 1995. [Online]. Available: <https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic>
- [4] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA: O’Reilly Media, 2022.
- [5] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.
- [6] Scikit-learn Developers, “Decision Trees,” *Scikit-learn Documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/tree.html>
- [7] Scikit-learn Developers, “Ensemble Methods,” *Scikit-learn Documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/ensemble.html>